EJERCICIOS PRÁCTICOS

WEB CRAWLER

Daniel Blanco Calviño

WEB CRAWLER

- Sistema que navega por la web con el objetivo de descubrir nuevo contenido.
  - Buscadores web.
  - Procesamiento de contenido específico.
  - Monitorización de páginas web.

REQUISITOS

- **Requisitos funcionales.**
  - Procesamiento de páginas web nuevas y editadas.
  - Procesamiento de cualquier tipo de contenido. HTML, PDF e imágenes.
  - Almacenar el contenido durante 10 años.

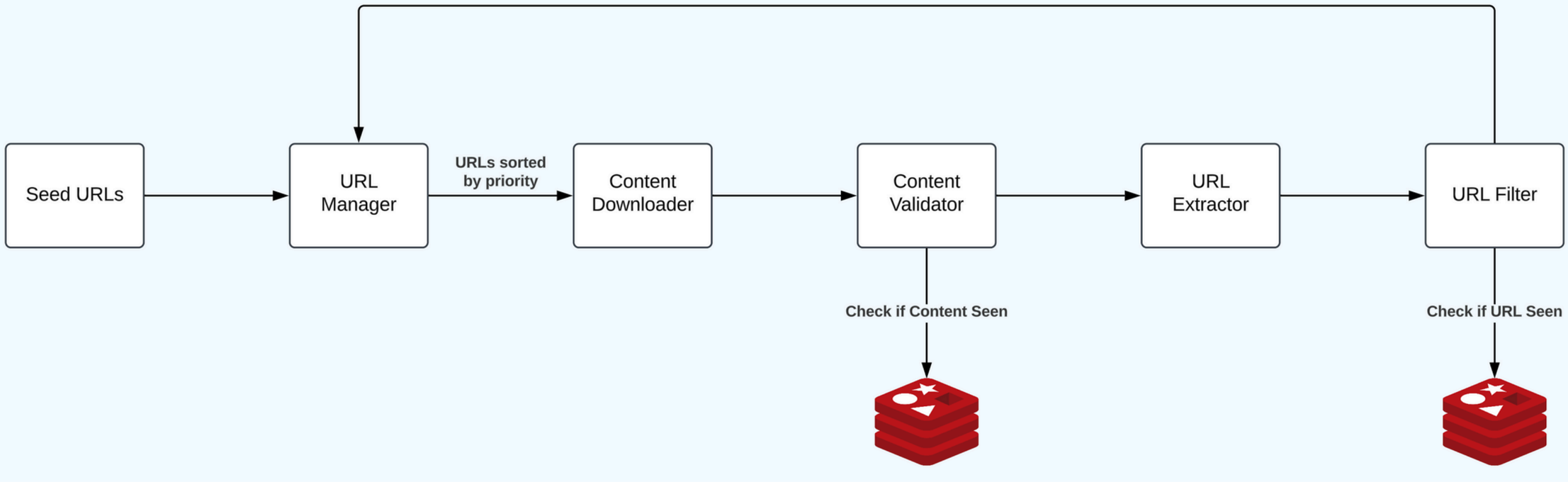
- **Requisitos no funcionales.**
  - Alta escalabilidad y disponibilidad. Millones de páginas web.
  - Tolerancia a fallos.
    - Reintentar webs fallidas.
    - Robustez ante problemas. Servidores caídos, tiempo de respuesta alto etc.
  - Procesar sólo sitios que lo permitan y no saturar servidores.

HIPÓTESIS Y ESTIMACIONES

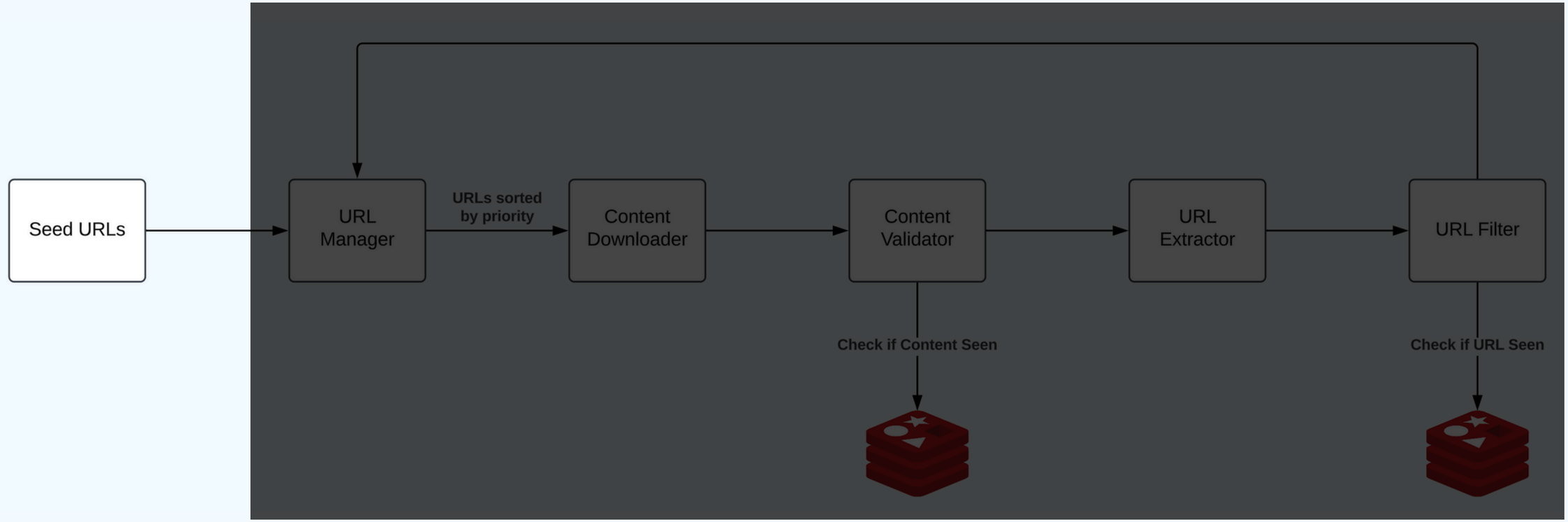
- Procesamiento de 2 mil millones de páginas al mes.
  - **770 páginas por segundo.**

- Tamaño medio de cada página o archivo de 1 MB.
  - Almacenamiento necesario para 10 años: **243 Petabytes.**

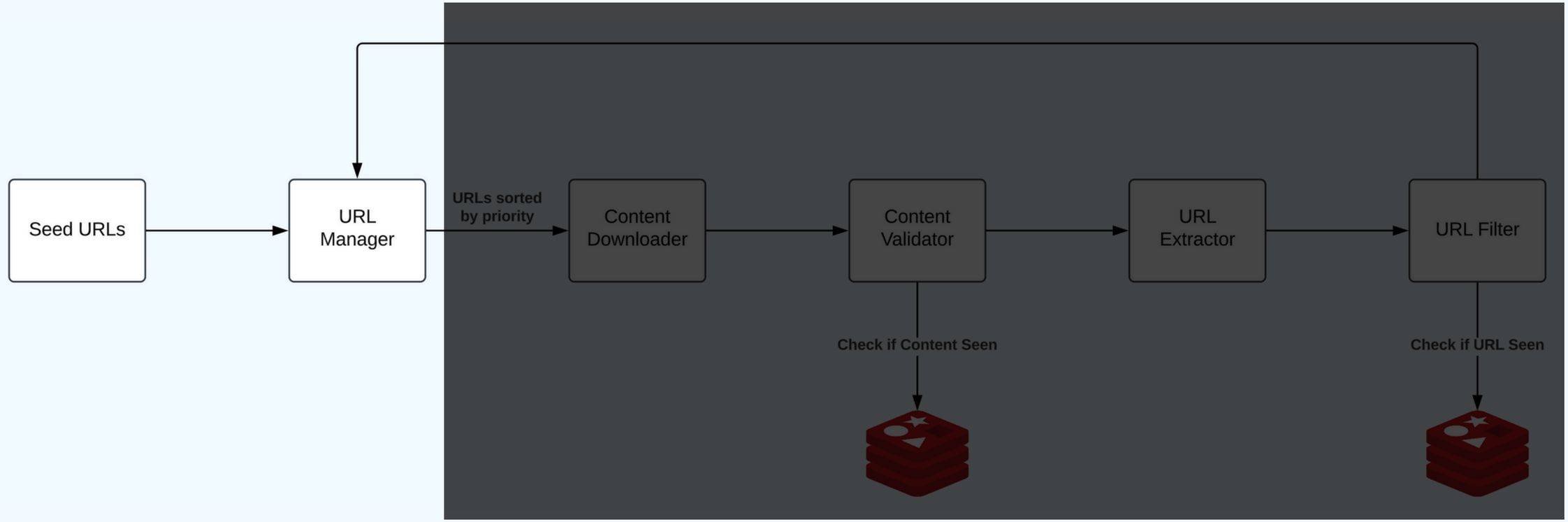
DISEÑO ALTO NIVEL



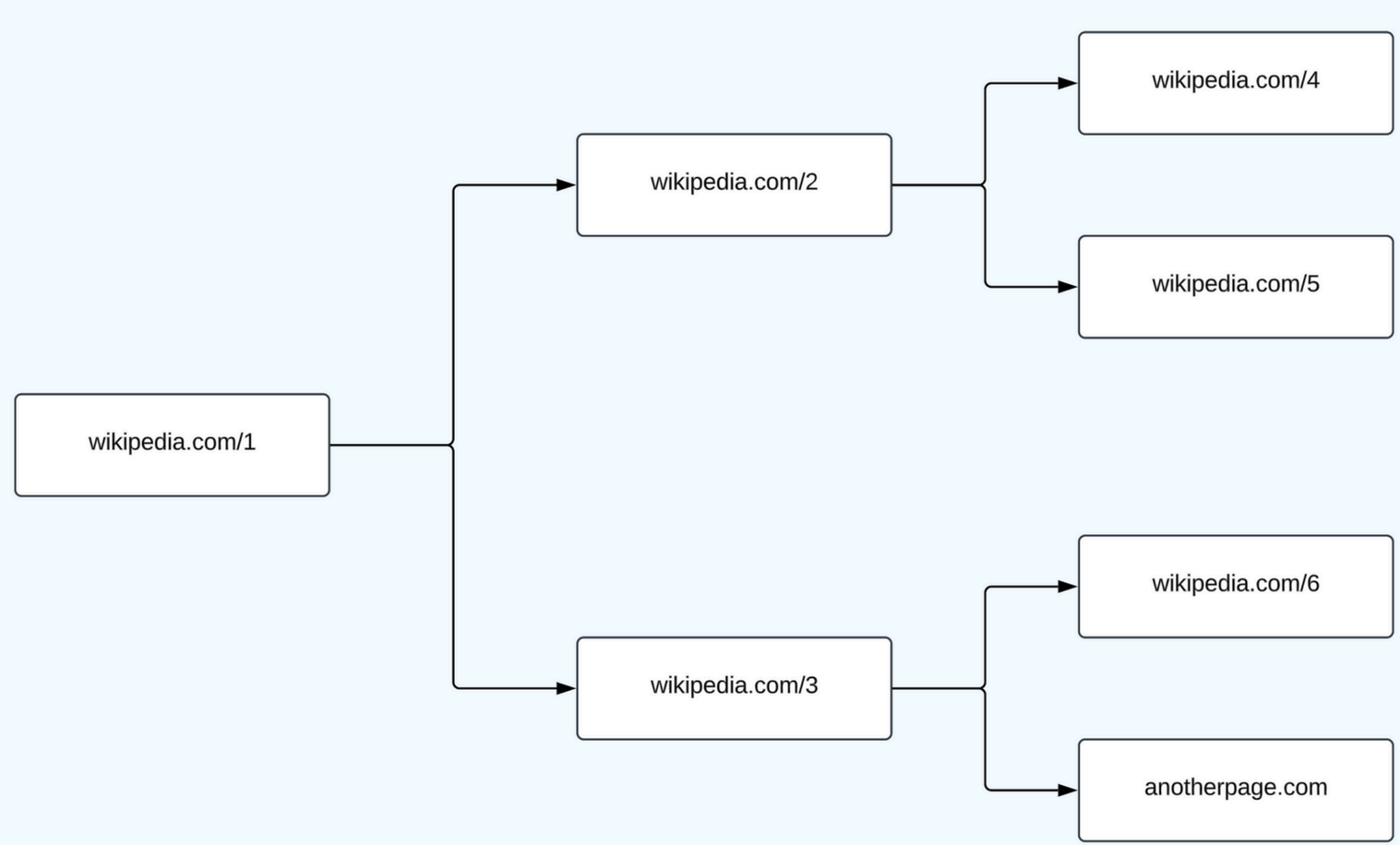
DISEÑO ALTO NIVEL



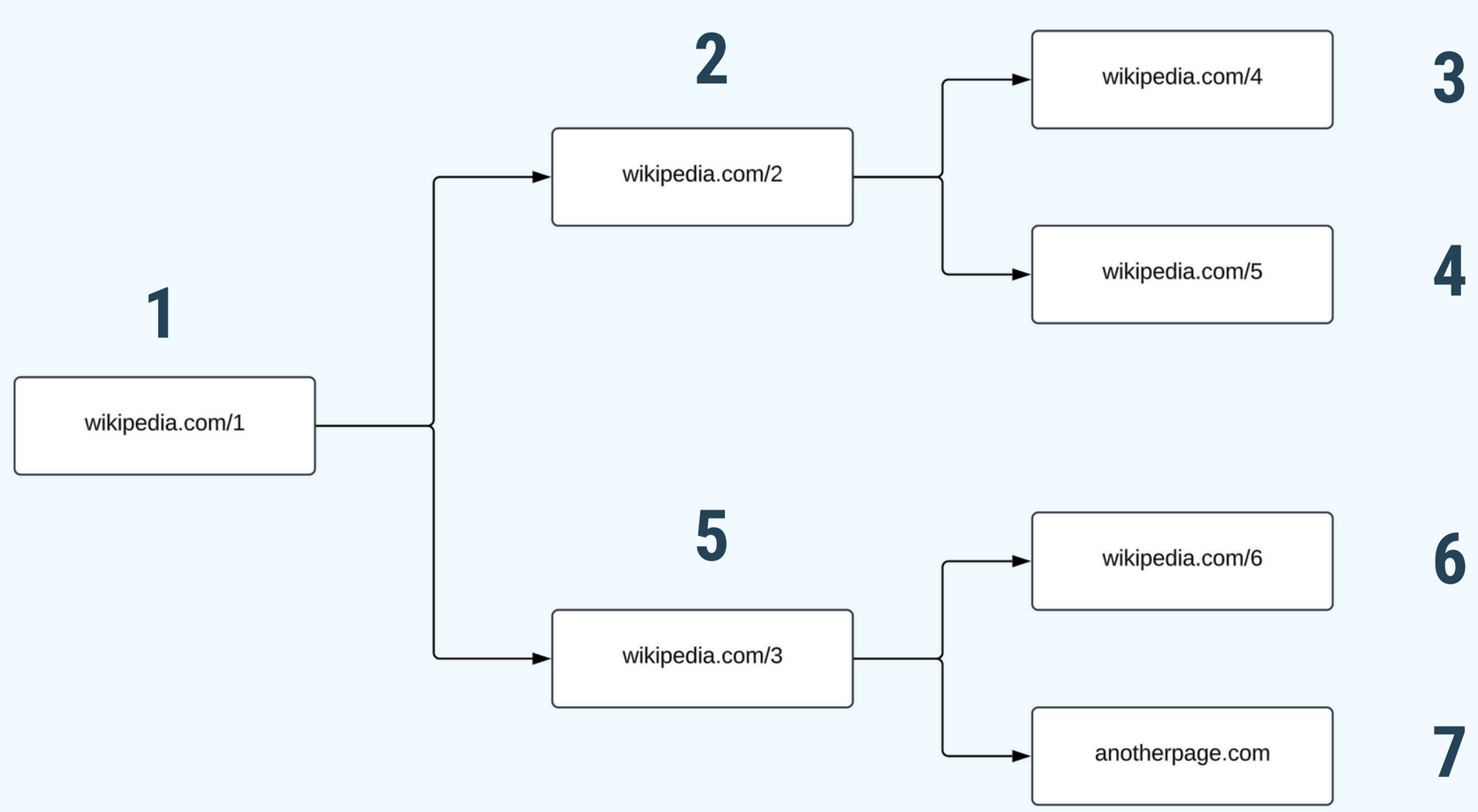
DISEÑO ALTO NIVEL



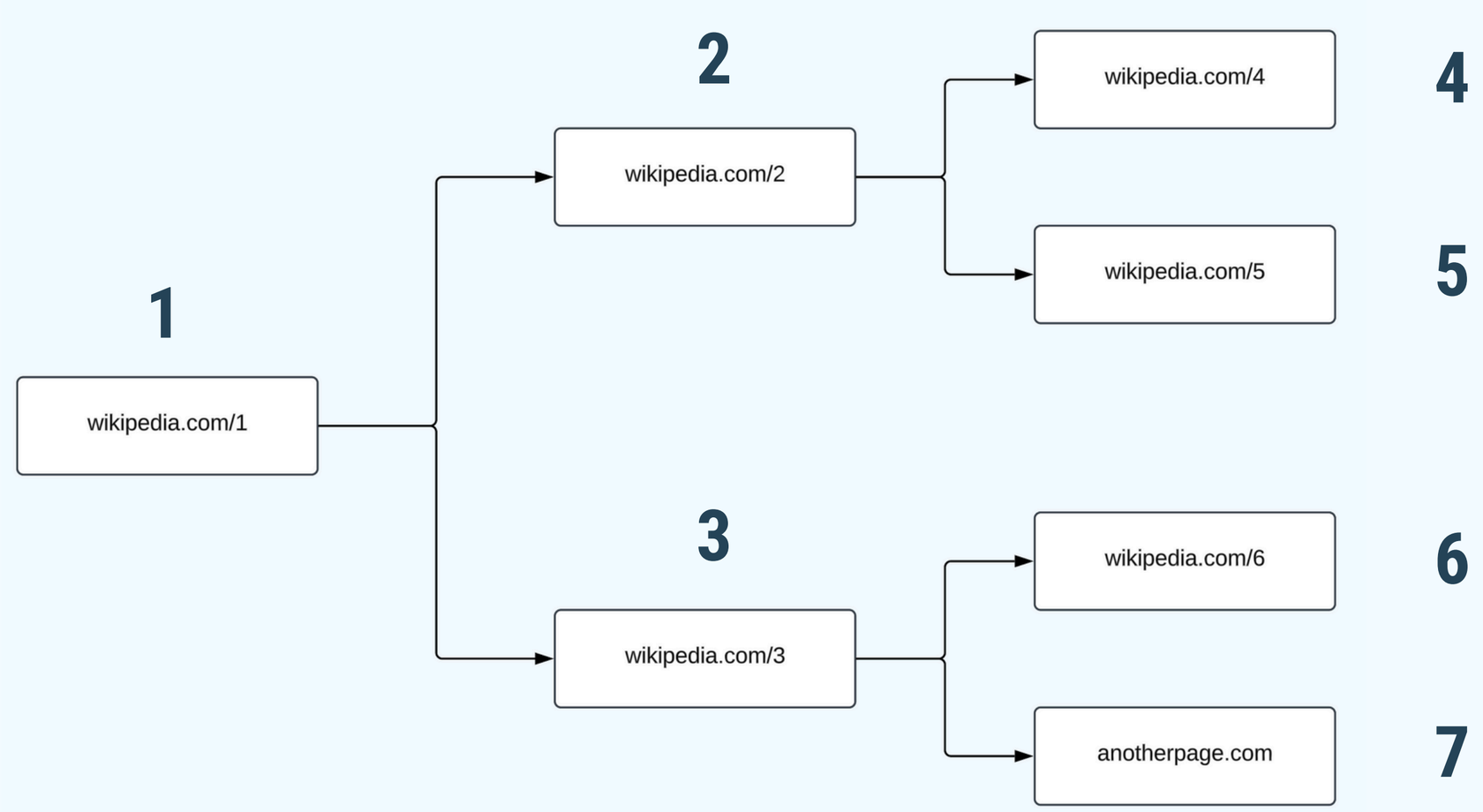
LA WEB ES UN GRAFO DIRIGIDO



BÚSQUEDA EN PROFUNDIDAD



BÚSQUEDA EN AMPLITUD

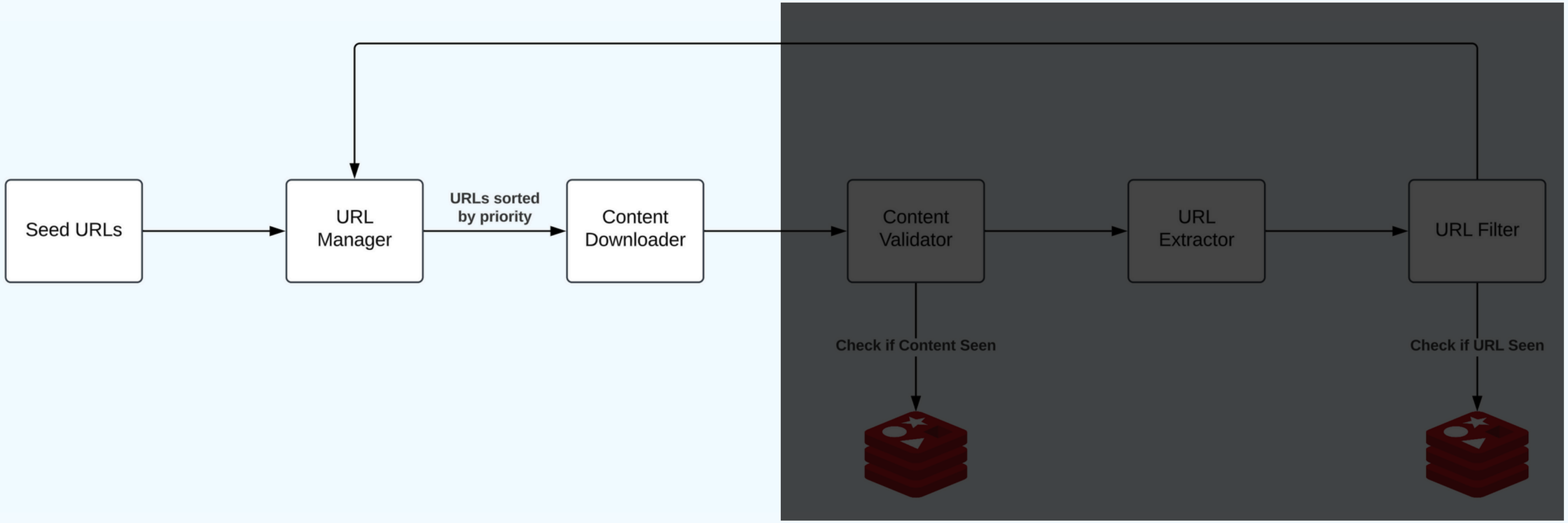


PROFUNDIDAD O AMPLITUD

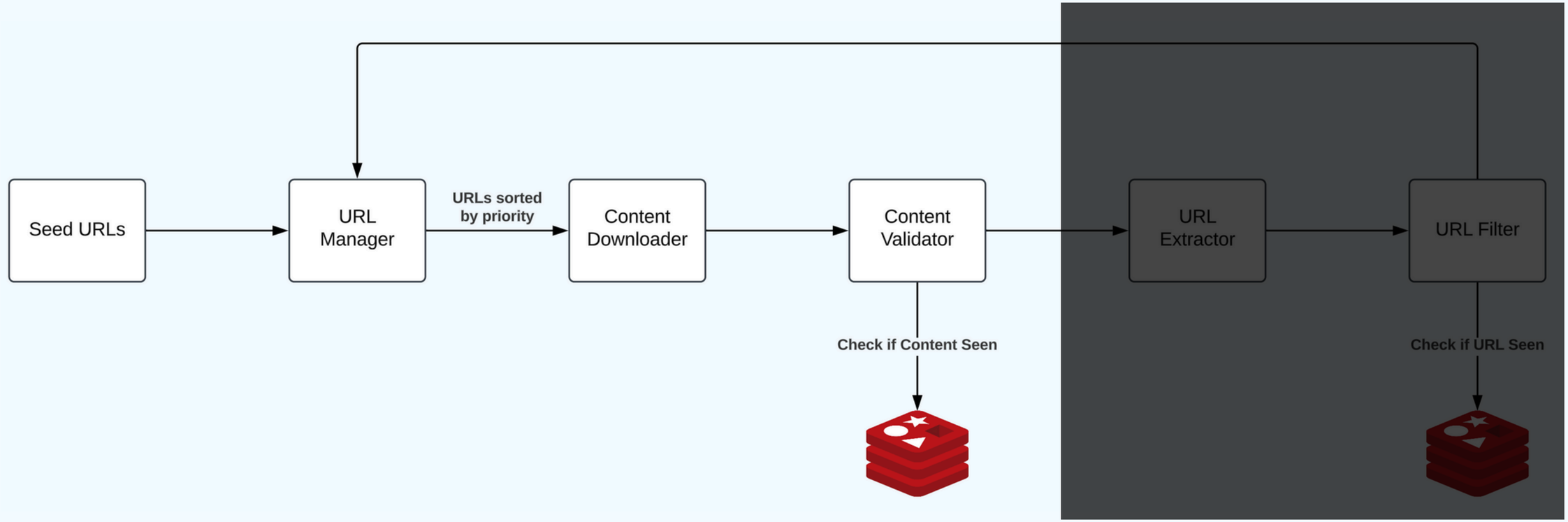
- **Búsqueda en profundidad.**
  - Visita los nodos hijos antes de visitar a los hermanos.
  - Casi todas las páginas tienen enlaces.
    - Problemático, ya que se alcanzaría una **profundidad enorme.**
      - Se podría imponer una profundidad límite.
  - Prioriza el **primer enlace disponible.**
    - No es posible priorizar por relevancia u otros parámetros.

- **Búsqueda en amplitud.**
  - Procesa todos los hermanos antes de pasar a los hijos.
    - Facilita la priorización por otros parámetros.

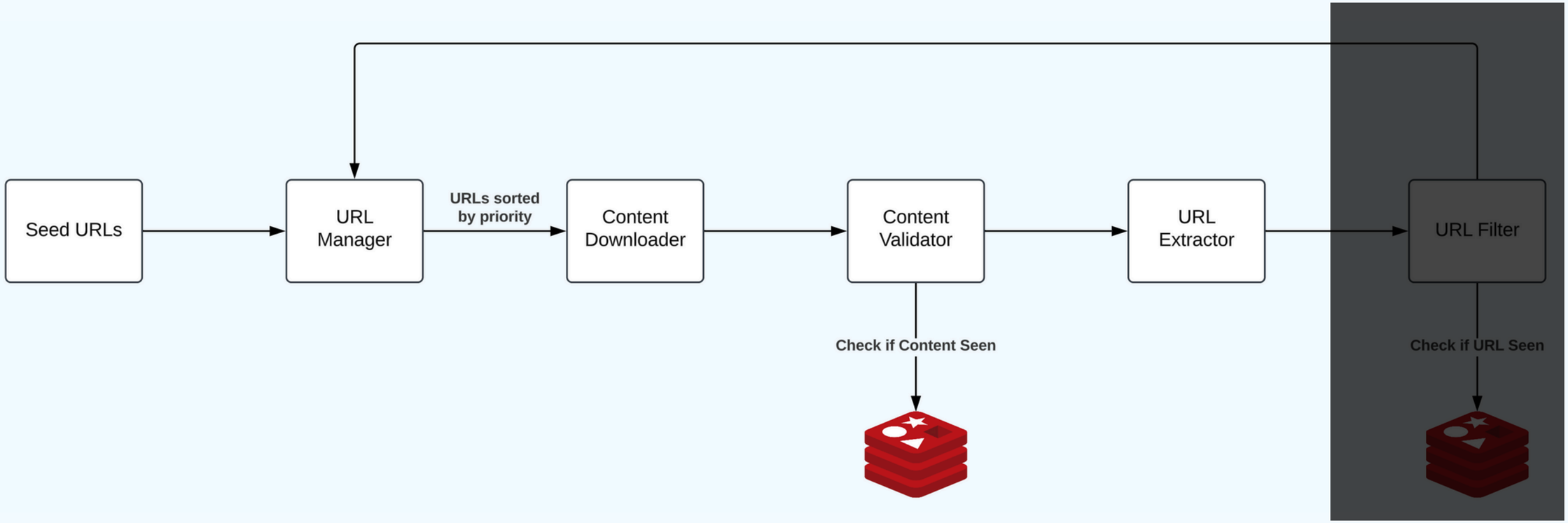
DISEÑO ALTO NIVEL



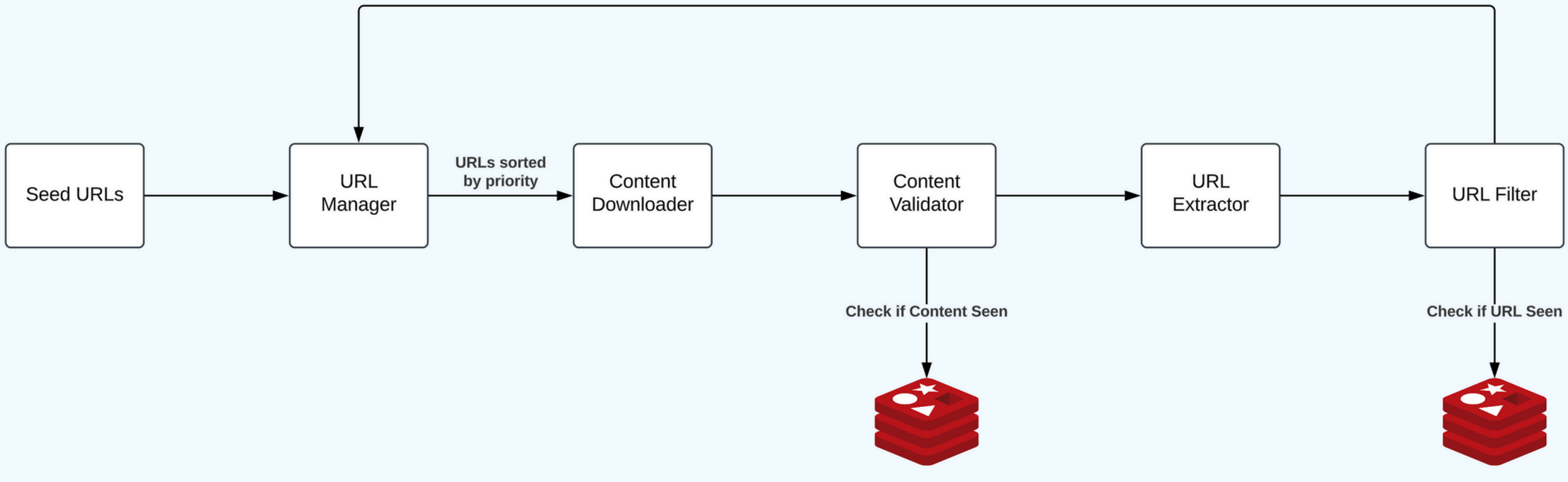
DISEÑO ALTO NIVEL



DISEÑO ALTO NIVEL



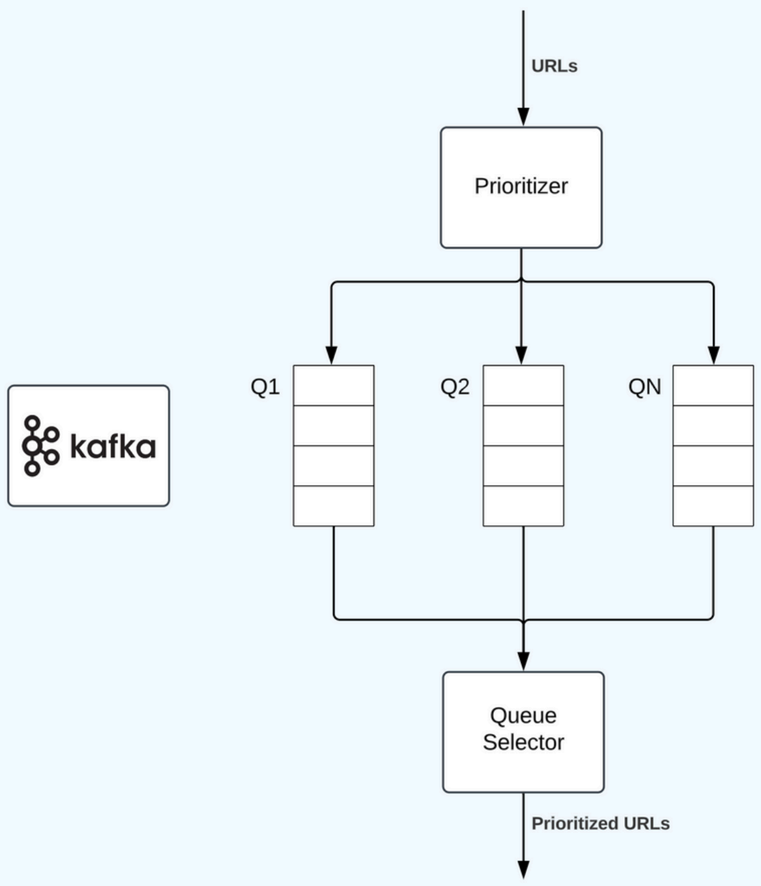
DISEÑO ALTO NIVEL



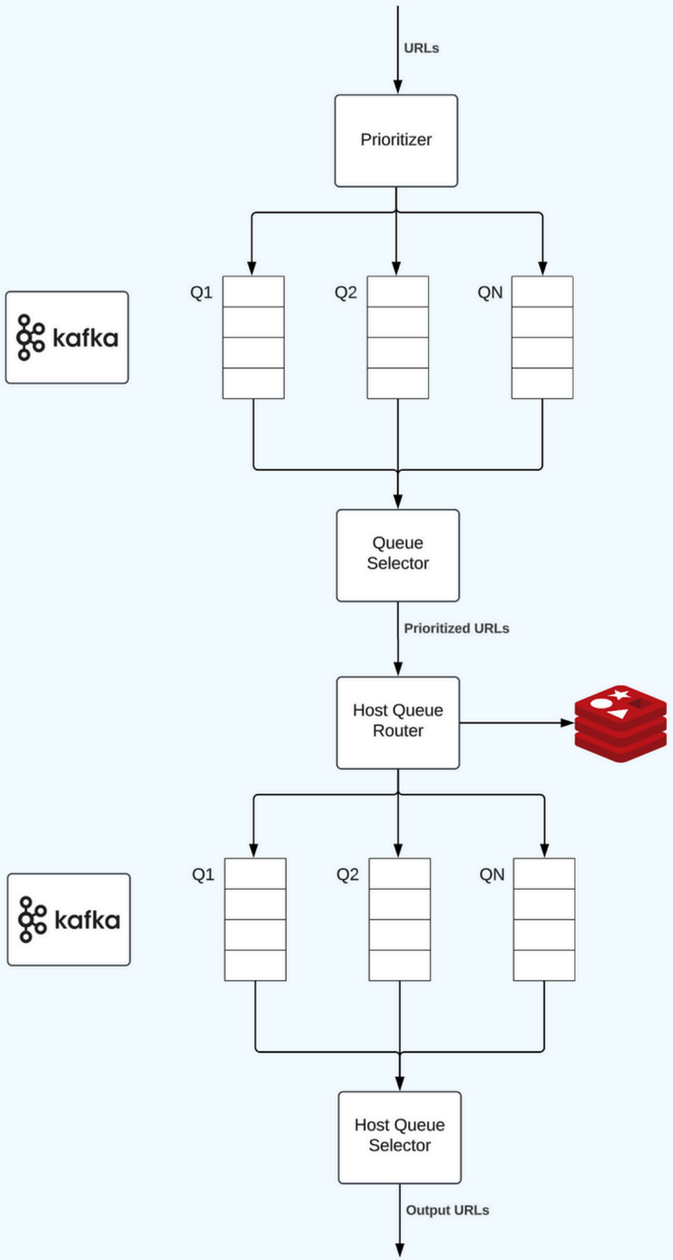
URL MANAGER EN DETALLE

- **Prioritizer.**
  - Asigna una prioridad a cada URL de entrada.
    - Escala de 1 a N.
    - Se asigna cada URL a una cola.

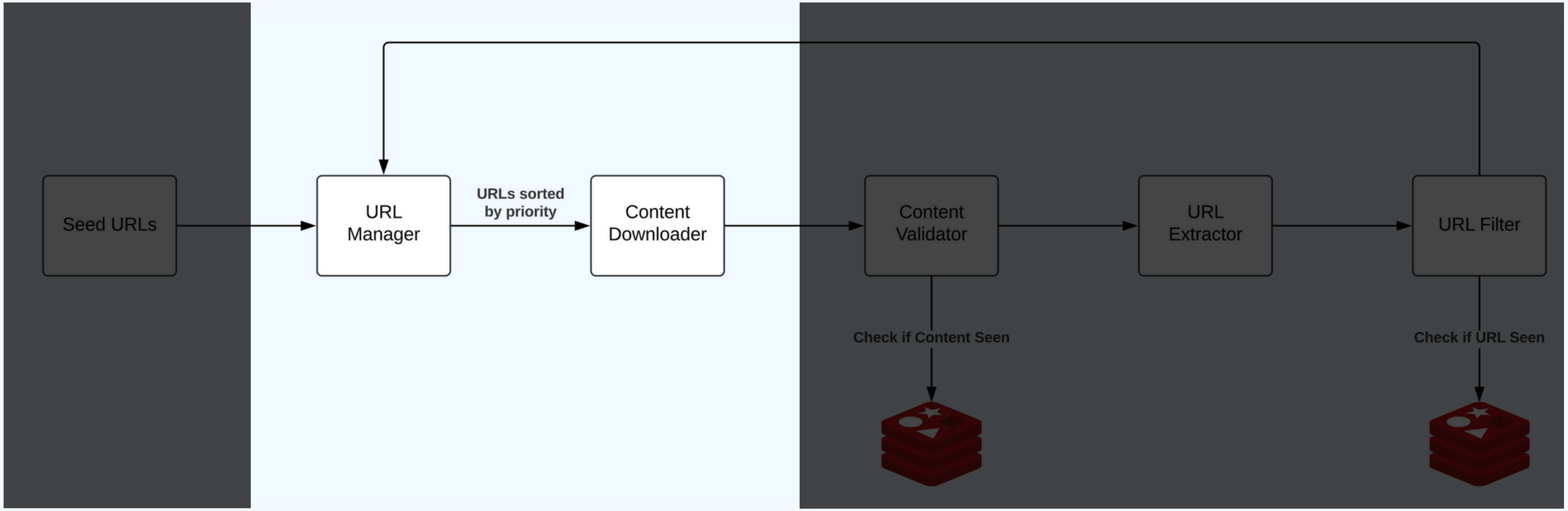
- **Queue Selector.**
  - Selecciona y consume URLs de las colas en función de la prioridad.



- **Prioritizer.**
  - Asigna una prioridad a cada URL de entrada.
    - Escala de 1 a N.
    - Se asigna cada URL a una cola.
- **Queue Selector.**
  - Selecciona y consume URLs de las colas en función de la prioridad.
- **Host Queue Router.**
  - Asigna cada host a una cola.
- **Host Queue Selector.**
  - Una única URL de cada host al mismo tiempo.



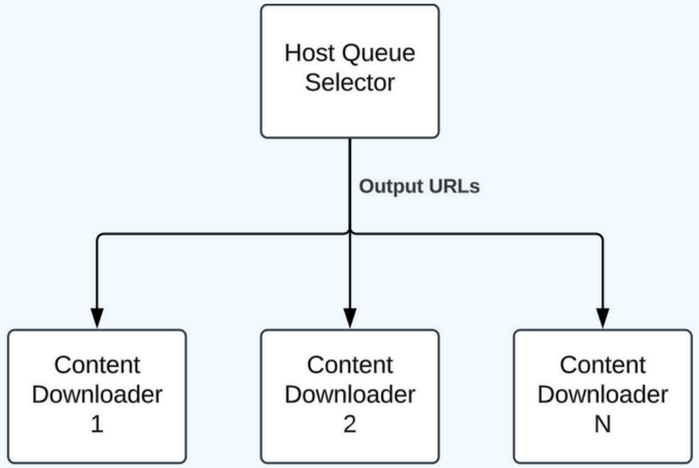
CONTENT DOWNLOADER EN DETALLE



CONTENT DOWNLOADER EN DETALLE

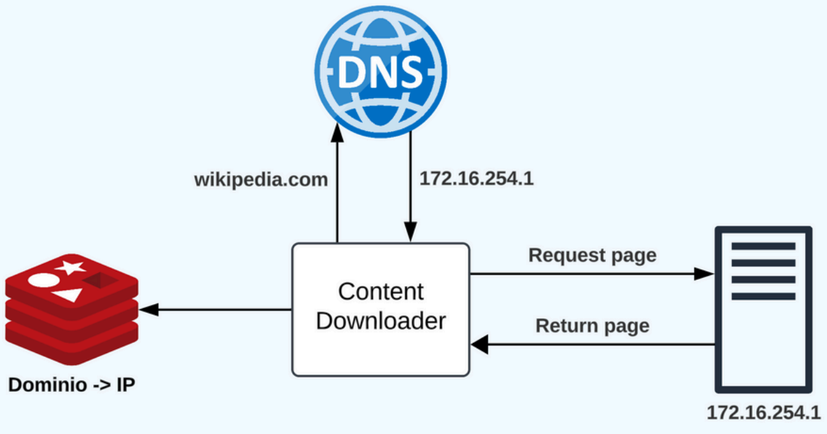
- **Escalado horizontal.**
  - Podemos desplegar tantas instancias como nos sea necesario.

- **Escalado dinámico.**
  - Desplegar o eliminar instancias dependiendo del número de URLs pendientes en las colas.



CONTENT DOWNLOADER EN DETALLE

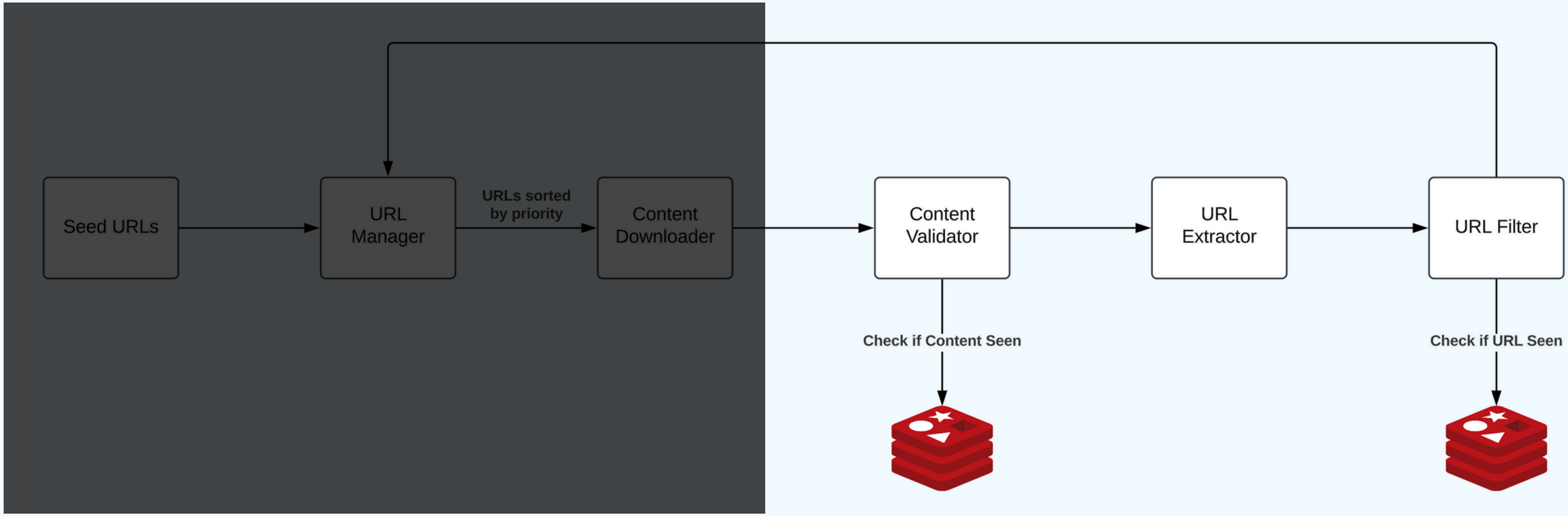
- **Cuello de botella: DNS**
  - Necesitamos consultar la dirección IP asociada a un dominio.
    - Esto añade más tiempo al componente más lento del crawler.
  - Solución: **cachear las direcciones IP** de los dominios que procesemos.
    - Si ya tenemos la IP para un dominio, la usamos sin pasar por el DNS.
    - Si no está o falla, reintentamos haciendo una petición al DNS.



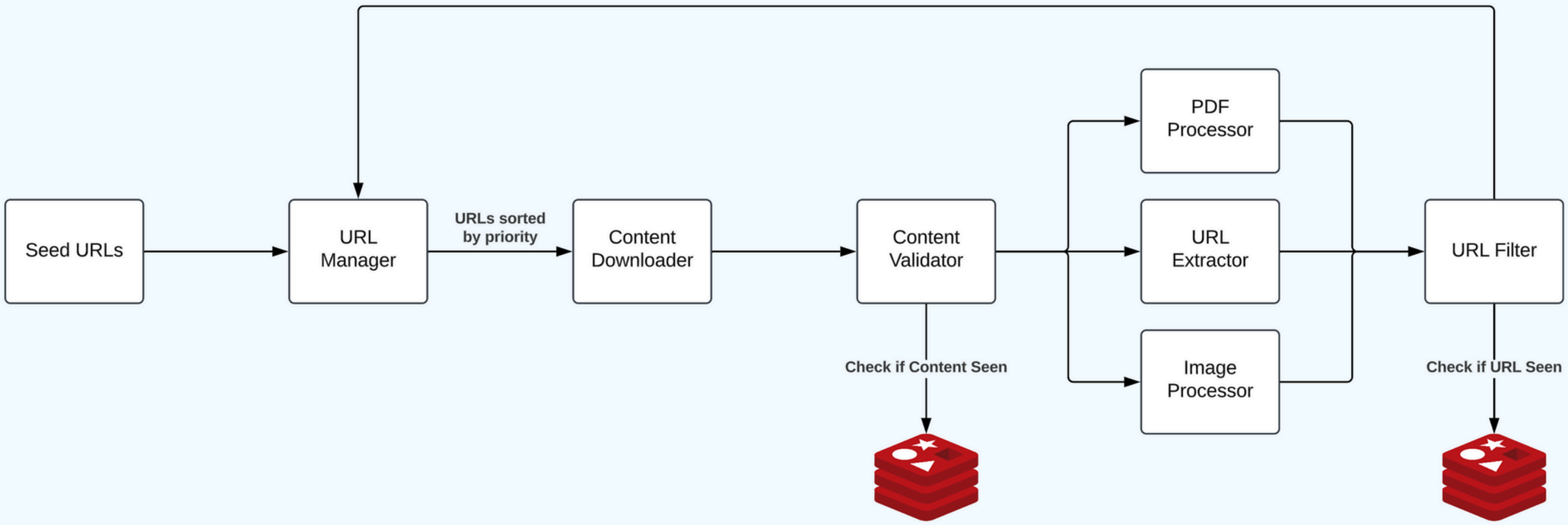
CONTENT DOWNLOADER EN DETALLE

- **Establecer timeouts bajos.**
  - Las páginas que tardan demasiado en responder ralentizarán mucho el proceso de crawling.
  - Timeouts de uno o dos segundos.
    - Si una web no responde, se puede reintentar más tarde con un timeout ligeramente mayor.

URL EXTRACTOR



MÚLTIPLES TIPOS DE FICHEROS



MÁS OPTIMIZACIONES

- **Servidores en múltiples regiones geográficas.**
  - Descarga de contenido desde servidores más cercanos.
    - **Mejora la latencia**.
  - Permite **saltarnos restricciones** impuestas en ciertos países.
    - Si un país bloquea una página, no podríamos procesar su contenido si todos nuestros servidores están situados allí.
    - Con servidores en múltiples países, podemos evitar este problema.

MÁS OPTIMIZACIONES

- **Servidores en múltiples regiones geográficas.**
  - Descarga de contenido desde servidores más cercanos.
    - **Mejora la latencia**.
  - Permite **saltarnos restricciones** impuestas en ciertos países.
    - Si un país bloquea una página, no podríamos procesar su contenido si todos nuestros servidores están situados allí.
    - Con servidores en múltiples países, podemos evitar este problema.

- **Gestión del fichero robots.txt**
  - Define las reglas que los crawlers deben seguir al acceder a un sitio web.
  - Podríamos cachear el fichero.

MÁS OPTIMIZACIONES

- **Almacenamiento eficiente.**
  - Agrupar contenido por prioridad.
    - Datos recientes o que se acceden frecuentemente.
    - Datos históricos o menos relevantes.
      - Almacenamiento en frío para ahorrar costes.
  - Compresión de contenido poco accedido.

MÁS OPTIMIZACIONES

- **Almacenamiento eficiente.**
  - Agrupar contenido por prioridad.
    - Datos recientes o que se acceden frecuentemente.
    - Datos históricos o menos relevantes.
      - Almacenamiento en frío para ahorrar costes.
  - Compresión de contenido poco accedido.

- **Server-side rendering.**
  - Muchas webs utilizan JS y librerías para renderizar contenido dinámico.
    - Si sólo descargamos el HTML estaríamos perdiendo contenido.
  - Utilizar server-side rendering en entornos aislados y sólo cuando sea necesario.